

QUESTION

What is **recursion**? Describe it with the line analogy: what do you ask, who answers, and what do you do with the answer?

QUESTION

Name the **three parts** of a recursive solution.

QUESTION

What is the **leap of faith**? What are you allowed to assume, and what must you still write?

QUESTION

```
int sumTo(int n) { return n +  
sumTo(n - 1); }
```

What happens when you call `sumTo(3)`?
What one line fixes it?

QUESTION

Trace `factorial(3)`. In what order do the calls **start**, in what order do they **finish**, and what does each return?

QUESTION

How many **stack frames** exist at the deepest point of `factorial(4)`? Does each frame have its own `n`?

ANSWER

- 1. **Base case:** solve the smallest problem directly.
 - 2. **Recursive case:** move one step closer to the base case by calling the function on a smaller problem.
 - 3. **Combine:** after the recursive call, use its result (or finish the work still needed) to solve the original problem.
- For factorial: base `n == 0`; recursive `factorial(n-1)`; combine `n *`.

ANSWER

It **never stops**: there is no base case. The calls go `sumTo(3)`, `(2)`, `(1)`, `(0)`, `(-1)`, ... until the call stack runs out of space: a **stack overflow**, usually a crash.

Fix: put `if (n == 0) { return 0; }` **before** the recursive call.

ANSWER

Five: `factorial(4)`, `(3)`, `(2)`, `(1)`, `(0)`. The base-case call has its own frame.

Yes: every call has its **own frame** with its own `n` (`4`, `3`, `2`, `1`, `0` all exist at once).

n = 0	newest (top)
n = 1	
n = 2	
n = 3	
n = 4	oldest (bottom)

ANSWER

A function that solves a problem by calling *itself* on a smaller version of the same problem.

Line analogy: “How many people are in front of me?” Ask the person ahead; they ask the one ahead of them. The first person in line knows the answer (`0`). Each person adds `1` to the answer they receive and passes it back.

ANSWER

Assume the recursive call already works for every case closer to the base case.

Your job is only to (1) handle the base case, and (2) build the answer for this case from the answer to the smaller case.

Do not trace every call in your head while writing the code.

ANSWER

Start: `factorial(3)`, `(2)`, `(1)`, `(0)`.

Finish: `(0)` returns `1`, `(1)` returns `1 · 1 = 1`, `(2)` returns `2 · 1 = 2`, `(3)` returns `3 · 2 = 6`.

The last call to start is the first to finish: **last in, first out**.

QUESTION

How is the **call stack** related to the **stack ADT**? Which call finishes first?

QUESTION

What does `show(2)` print?

```
if (n == 0) return;
cout << n << ' '; (A)
show(n - 1);
cout << n << ' '; (B)
```

QUESTION

`f(n)` has base case `n == 0` and calls `f(n - 2)`. Why is `f(5)` a problem? Give a better base case.

QUESTION

Write `printReverse(std::string s)` using `s.substr(1)`. Which line is the base case, and which is the combine step?

QUESTION

Version B adds a second parameter, `i` (type `size_type`). What does `i` mean? Why is B more efficient than A?

QUESTION

`sumArray(a, n)` returns 0 when `n == 0`, else `a[n - 1] + sumArray(a, n - 1)`. For `a = {2, 5, 3}`, what does `sumArray(a, 3)` return? How many calls?

CLASS 11 • RECURSION • BEFORE AND AFTER8 ANSWER 2 1 1 2 Line (A) runs on the way down: 2, then 1. Line (B) runs on the way back up, after each deeper call returns: 1, then 2. <i>Printing after the recursive call reverses the order. That is the idea behind <code>printReverse</code>.</i>	CLASS 11 • RECURSION • CALL STACK7 ANSWER A call is like a push of a frame; a return is like a pop. The call stack is last in, first out: the most recent call that has not returned finishes first. The computer maintains this stack for you. <i>Class 9: the bracket matcher only looks at the top. A recursive call only needs the newest frame.</i>
CLASS 11 • RECURSION • PRINTREVERSE A10 ANSWER <pre>void printReverse(std::string s) { if (s.empty()) { return; } printReverse(s.substr(1)); std::cout << s[0]; }</pre> Base case: <code>s.empty()</code>. Combine: print <code>s[0]</code> <i>after</i> the call. <code>printReverse("abc")</code> prints cba. <code>s.substr(1)</code> is <code>s</code> without its first character.	CLASS 11 • RECURSION • PROGRESS9 ANSWER <code>n</code> goes 5, 3, 1, −1, −3, ... It skips over 0, so the base case is never reached and the stack overflows. Every recursive call must make progress toward a base case. Use <code>n <= 0</code> as the base case. <i>“Progress” does not always mean “smaller by 1.”</i>
CLASS 11 • RECURSION • ARRAY SUM12 ANSWER 10, using 4 calls (<code>n = 3, 2, 1, 0</code>). $3 + (5 + (2 + 0)) = 10$ The base case returns 0, the sum of no numbers. It does not change the total when it is combined with +.	CLASS 11 • RECURSION • PRINTREVERSE B11 ANSWER <code>i</code> is the position of the first character not yet handled. Base case: <code>i == s.length()</code>. Call it as <code>printReverse(s, 0)</code>. <pre>printReverse(s, i + 1); std::cout << s[i];</pre> Version A builds a new, shorter string in every call (about $n-1, n-2, \dots$ characters copied). Version B copies nothing: <code>s</code> is passed by reference and only <code>i</code> changes.

CLASS 11 • RECURSION • LOOP OR RECURSION? 13 QUESTION Factorial, reversal and sum can all be written with a <code>for</code> loop. When is recursion still worth it?	CLASS 11 • RECURSION • STACK OVERFLOW 14 QUESTION What limits how deep a recursion can go? Why can a recursion with n active calls crash even when it is correct?
CLASS 11 • RECURSION • DFS RULES 15 QUESTION State the recursive DFS helper for Project 2, in order: what happens when it is called on a cell?	CLASS 11 • RECURSION • ANSWER STACK 16 QUESTION In DFS, which cell is pushed onto the answer stack first? Which is on top at the end? When does each push happen?
CLASS 11 • RECURSION • BACKTRACKING 17 QUESTION A DFS call is not at the end, and every open, unvisited neighbor has failed. What happens? Do visited marks get erased?	CLASS 11 • RECURSION • DFS VS. BFS 18 QUESTION On the 4×4 Project 2 example, DFS finds a route of 9 cells and BFS finds one of 7. What does that tell you? Which container does each use?

CLASS 11 • RECURSION • STACK OVERFLOW 14 ANSWER Each active call keeps a frame on the call stack, and the stack has limited space. If n recursive calls are active at the same time, there are n frames. A correct recursion with a very large n can still overflow the stack. <i>Missing base case, or no progress toward it, gives the same crash.</i>	CLASS 11 • RECURSION • LOOP OR RECURSION? 13 ANSWER A loop is usually cheaper: a recursive call uses a stack frame; a loop does not. Recursion is worth it when the problem is naturally the same procedure applied to a part of it: a tree (each subtree is a tree), a maze search (“explore a neighbor the same way”). <i>Cost to remember: depth.</i>
CLASS 11 • RECURSION • ANSWER STACK 16 ANSWER The end is pushed first, when it is reached. Each caller then pushes <i>itself</i> when a recursive call came back with success. So the route is built as the calls unwind: end first, start last (on top). Popping the stack gives the route from start to end. <i>The same last-in, first-out idea, used twice: the call stack and the answer stack.</i>	CLASS 11 • RECURSION • DFS RULES 15 ANSWER <pre> helper(cell): mark cell visited if cell is the end: push cell on answer stack; return for each neighbor: left, right, up, down (open, not visited): helper(neighbor) if answer stack not empty: push cell; return </pre>
CLASS 11 • RECURSION • DFS vs. BFS 18 ANSWER DFS follows one branch as deep as it can and returns the first route it finds, not necessarily the shortest. BFS explores in layers, so its route has the fewest cells. DFS: a stack (the call stack, or an explicit one). BFS: a queue. <i>BFS uses the queue from Class 10.</i>	CLASS 11 • RECURSION • BACKTRACKING 17 ANSWER The call returns without finding a route (a dead end). Its caller then tries its next neighbor: that return-and-try-another is the backtrack. Visited cells stay visited. Never un-mark them, or the same dead end would be explored again and again.